Textbook: Wave Function Collapse Introduction

The Tile-Based Wave Function Collapse (WFC) algorithm is a powerful procedural generation method used to create structured, grid-based layouts. Unlike bitmap WFC, which deals with pixel patterns, tile-based WFC works with distinct tiles (like puzzle pieces), each having specific rules about how they can connect to neighboring tiles. It's widely used in game development for generating levels, maps, and other structured layouts.

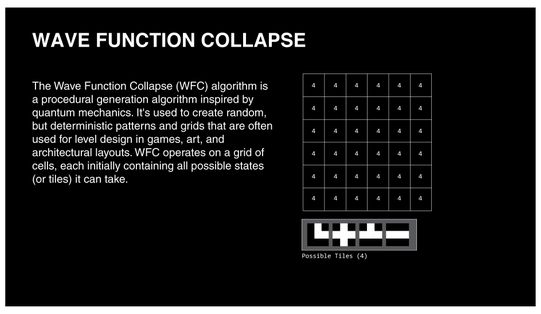
Key Concepts of Tile-Based WFC

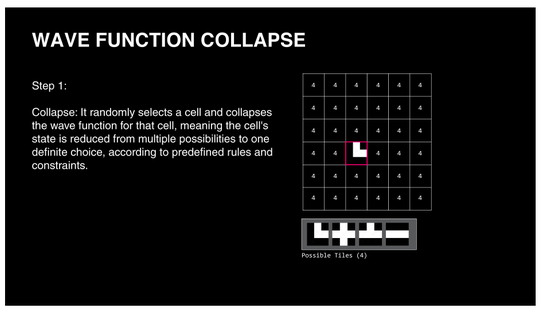
1. **Tiles**: Predefined blocks or elements with specific connection rules. Each tile can only connect to certain other tiles according to these rules.
2. **Grid and Superposition**: The algorithm operates on a grid, where each cell starts in a state of superposition, meaning it can potentially become any tile.
3. **Observation and Collapse**: A cell's superposition state is "observed" (or collapsed) to a single definite tile based on constraints and probabilities.
4. **Propagation**: This observation affects the state of neighboring cells, limiting their possible states to maintain coherence with the observed cell.

Implementing Tile-Based WFC in Processing

1. **Define Tiles and Rules**:
   - Create a set of distinct tiles.
   - Define rules for how tiles can connect (e.g., road tiles must connect to other road tiles).
2. **Initialize the Grid**:
   - Create a grid where the algorithm will run, with each cell starting with all possible tiles.
3. **Main Algorithm Loop**:
   - **Collapse**: Select a cell and collapse its state to a single tile, chosen randomly to start.
   - **Propagation**: Update neighboring cells, removing tile options that violate the rules.
   - Repeat this process until all cells are collapsed or no valid moves remain.

This would be the setup:





With our implementation, the collapse function looks as follows:

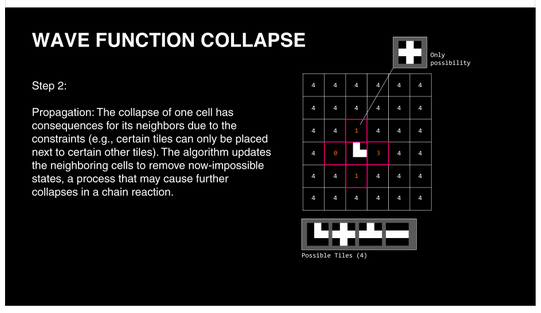
```
def collapse_cell(self, x, y):
    # Get the list of possible tiles for the cell at position (x, y)
    possibilities = self.cells[x][y]

    # Randomly select one tile from the list of possibilities
    # This tile will be the state the cell collapses into
    chosen_tile = random.choice(possibilities)

    # Update the cell's state to be just the chosen tile
    # The cell's superposition is now collapsed to this single state
    self.cells[x][y] = [chosen_tile]

    # Call the propagate method to update adjacent cells based on the collapse of this cell
    # Propagation is necessary to maintain the coherence of the overall pattern
    self.propagate(x, y)

```



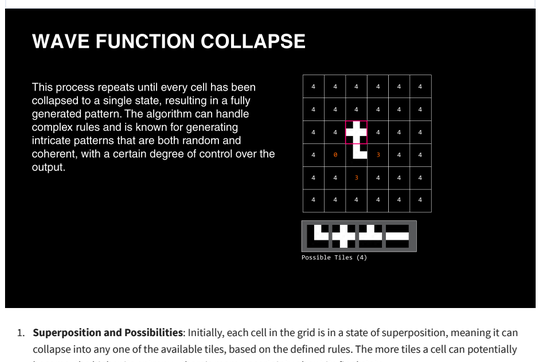
This is how we propagate the tiles. Notice that the final function 'update_neighbors' starts addressing the issue of tile compatibility that we will discuss in the next textbook.

```
def propagate(self, x, y):
    # Retrieve the tile to which the cell at (x, y) has collapsed
    # At this point, the cell should have only one possible state left
    collapsed_tile = self.cells[x][y][0]

    # Define the coordinates of the neighboring cells
    # Neighbors are categorized as 'top', 'bottom', 'left', 'right'
    neighbors = {
        'top': (x, y - 1),
        'bottom': (x, y + 1),
        'left': (x - 1, y),
        'right': (x + 1, y)
    }

    # Iterate through each neighboring cell
    for direction, (nx, ny) in neighbors.items():
        # Check if the neighbor is within the grid boundaries
        if 0 <= nx < self.cols and 0 <= ny < self.rows:
            # Update the possible states of the neighboring cell based on the collapsed tile
            # This is typically done in a method like 'update_neighbors'
            # which would modify the neighbor's possible states to be consistent with 'collapsed_tile'
            self.update_neighbors(nx, ny, collapsed_tile, direction)


```



1. **Superposition and Possibilities**: Initially, each cell in the grid is in a state of superposition, meaning it can collapse into any one of the available tiles, based on the defined rules. The more tiles a cell can potentially become, the higher its entropy - there's more uncertainty about its final state.
2. **Measuring Entropy**: In WFC, entropy is often quantified as the number of possible states (tiles) a cell can still collapse into. A cell with many possible states has high entropy, while a cell with fewer possible states has low entropy.
3. **Entropy and Decision Making**: The algorithm often prioritizes cells with the lowest entropy for observation and collapse (excluding those already collapsed into a single state). The rationale is that these cells have the least uncertainty and making a decision here will have a clearer impact on the surrounding cells, helping to propagate constraints more effectively.

Tile-based WFC is an excellent algorithm for procedurally generating structured, rule-based layouts. The key challenge lies in defining a set of tiles and rules that can produce diverse yet coherent patterns or layouts. The algorithm is particularly effective in scenarios where structured design is essential, such as level design in games or layout generation in digital art applications.

Go to next item ✓ Completed

Like Dislike Report an issue